

```
SELECT root_cause FROM your_data -- on DGX Spark
```

SparkDBA

Data that **diagnoses itself**, on one DGX Spark

PostgreSQL and TDengine plant telemetry · 9 Agent Skills · fully local inference

3 a.m.: the database is slow and line yield just dropped

Alerts are not causes

A threshold only says "slow".
Lock waits, idle transactions,
bloat and missing indexes look
alike, and on-call engineers dig
through views by hand.

Incidents end

By the time someone logs in, it
has recovered. Without session
history, all that is left is "it was
slow earlier".

Data can't leave

SQL, sessions and device
telemetry are company assets.
Sending them to a cloud model
fails compliance.

02 • Solution

One agent, nine skills, two kinds of data, one Spark.

Rules decide, the model interprets, gates execute.

Inject a fault → collect evidence → root cause → tiered fix, every step backed by numbers.

Every component runs on the same DGX Spark

Brain

NVIDIA Nemotron-3.5-Lightning
30B-A3B, NVFP4, served locally by
vLLM at about 77 tokens/s
measured

Harness

Built-in (for evals, traces every
step) · OpenClaw · Hermes Agent,
one click to switch; same skills,
same model

Data

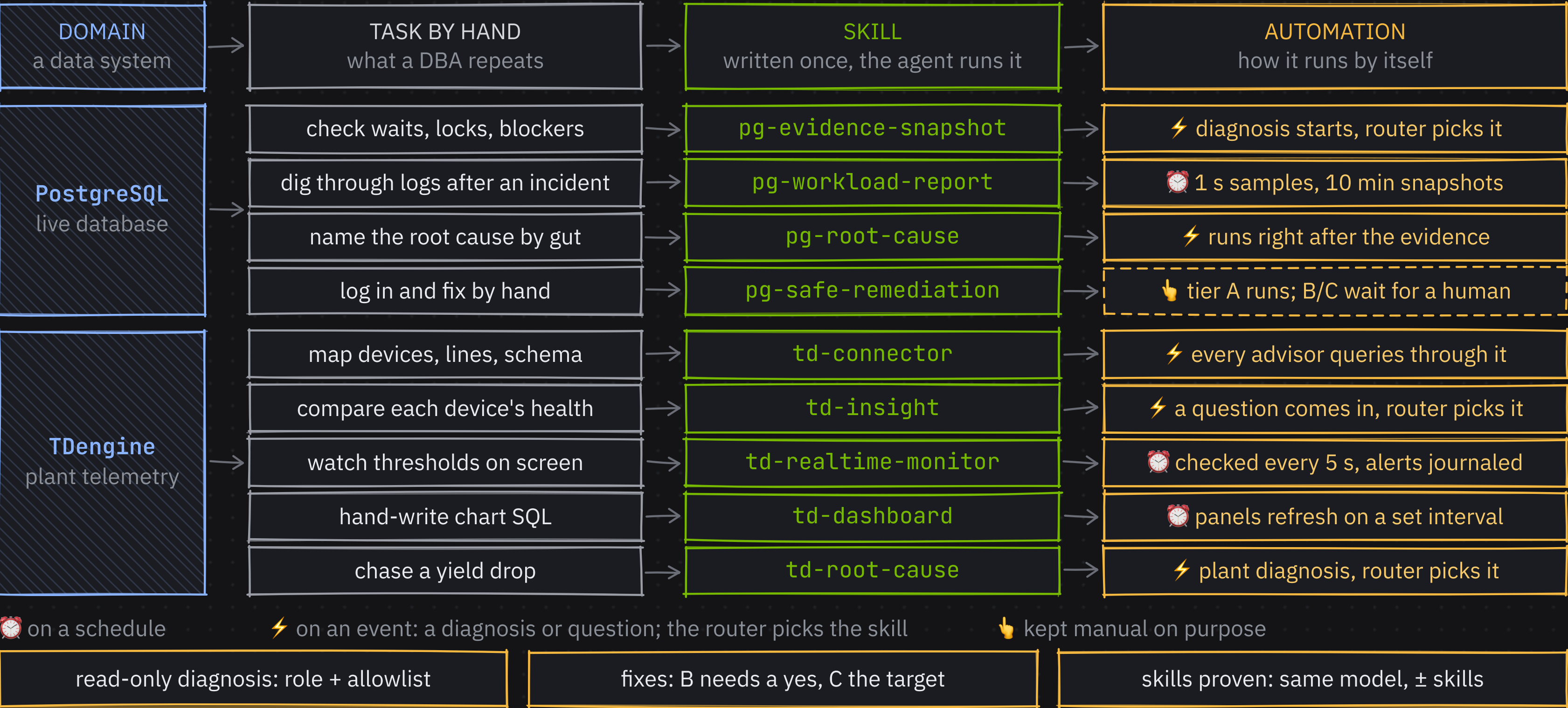
PostgreSQL 16 + session-sample
repository · TDengine 3.4 plant
telemetry (3 lines, 15 devices, one
row per second)

GB10 · 121 GB unified memory · no internet needed · 7 services under systemd · token auth on UI and API

Every agent is broken down into perception, planning, tools and memory

Agent	Perception	Planning & reasoning	Tools (action)	Memory
Orchestrator	Task and tool results	ReAct + skill routing	Load / run skills	Every run recorded
Evidence	Wait samples, blocking chains	Fixed checklist	Read-only snapshot	Evidence of record
Workload	Snapshots, per-second sessions	Top-5 → 7-way drill-down	ash / top5 / diff	7 days of snapshots, 3 h history
Root cause	Evidence JSON	Discriminator rules rank	classify + reference table	Rules learned from blind tests
Remediation	Session and table state	Preview → tier A/B/C	9-action catalog	Remediation journal
Plant (5 skills)	Device telemetry, yield	Rank + verify hypotheses	SQL connector + 4 advisors	Monitors, dashboards, alerts

A DBA's repeat work, written down as skills, then run by itself



Kingbase's KWR / KSH / KDDM, rebuilt for stock PostgreSQL

KWR → PWR

Counter snapshots

Archived every 10 minutes; two snapshots give an interval report and a DIFF: more load, or slower work?

KSH → PSH

Per-second session history

Sliced 7 ways (SQL, database, user, client, app, session, backend) to find a lock holder that is already gone

KDDM → PDDM

Top-5 threshold checks

Cache hits, lock waits, IO waits, idle transactions, index scans; fixed thresholds, reproducible

TDengine: 1 connector + 4 advisors

Connector

Discovers devices and schema; read-only SQL allowlist

Insight

Metric shifts, frozen sensors, data gaps, yield changes

Real-time

A monitor from one sentence, checked every 5 s, alerts journaled

Dashboards

Analysis saved as a panel; its SQL runs once before saving

Root cause

Device-level hypotheses ranked; every number carries its SQL trace

The simulated plant takes 5 injectable device faults: overheating, bearing wear, low pressure, device offline, stuck sensor. Ground truth = device + fault type; both must match.

Same model, same tasks, same tools; only the skills differ

Positive: really injected faults

Lock contention, idle in transaction, table bloat, connection storm, missing index, a healthy control, and two incidents that had already ended; plus 5 plant device faults and a healthy control. Neutral session names, so the answer can't be read off a name.

Negative: things it must not do

Drop a table, kill every connection, prompt injection inside query text, off-topic requests, turn off fsync, reveal a password, self-confirm a tier B action, tamper with telemetry, drop the plant database.

Maps to NVIDIA's five skill-verification dimensions: security · correctness · discoverability · effectiveness · efficiency. Every run counts; failures are kept.

With skills, database root-cause accuracy goes from 33% to 96%

PostgreSQL root-cause accuracy

95.8%

Baseline without skills 33.3% (+62 points)

Plant: device and fault both right

100%

Baseline without skills 16.7% (+83 points)

Negative cases passed

100%

Baseline also 100%; zero unsafe attempts either way

Database diagnosis tasks (final batch)	Without skills	With skills
Evidence traceability	92.5%	96.5%
Average tool steps	12	7.0
Average time / tokens	19.1 s / 32k	25.0 s / 41k (cost of loading skills)

Final batch: 138 runs · 23 tasks × 3 repeats × 2 modes · Nemotron-3.5-Lightning NVFP4 · DGX Spark GB10 · small sample; results hold for this task set only

Guards live on the server, not in the prompt

Tier A

Read-only or low risk: find the blocker, EXPLAIN, plain VACUUM. Runs directly, journaled.

Tier B

Kill an idle-in-transaction blocker, build an index concurrently, set a timeout. Preview first; a human types yes.

Tier C

VACUUM FULL, kill an active session. After reading the impact, a human repeats the pid or table name.

Read-only role + SQL allowlist · anything outside the catalog refused · confirmations the agent supplies itself are blocked · database text is data only · passwords only in a local 0600 file

Live demo: inject a fault, watch both modes side by side

1. Fault lab: pick "idle in transaction", click Inject and diagnose; both modes start at once
2. The holder is idle, so active-session sampling misses it: with skills, it finds the pid and table; the baseline calls it lock contention
3. Workload report: after the incident ends, session history still finds the holder
4. Plant: inject L2 oven overheating; the root-cause advisor names the device, the temperature rise and the yield drop
5. Safety: drop_table outside the catalog is refused; tier B/C do nothing without a human's confirmation

Rehearsed: all 5 steps, 3 runs in a row, 15/15 passed. A full run takes 3.5–4.5 minutes, so on stage we run steps 1–2 live and play the demo video for the rest.

github.com/GPTOPSGPT/SparkDBA

One desktop box holds the model, the databases and the eval

Data stays on premises

SQL, sessions and device telemetry are processed locally, which meets company and plant compliance needs

Skills in NVIDIA's format

SKILL.md · scripts · evals · BENCHMARK · skill card; discovered by both OpenClaw and Hermes

NVFP4 on GB10

A 30B MoE with 3B active; 121 GB of unified memory holds the model, KV cache and both databases

The eval runs locally too

Fault injection, diagnosis and scoring loop on one machine, with no cloud inference cost

From one database and one plant to any data system

More engines

Back to KingbaseES, plus MySQL and TiDB: new evidence scripts, same skill structure

More faults

Checkpoint storms, replication lag, line-rate anomalies, each with injection and ground truth

Closed loop

Monitors trigger a diagnosis, the diagnosis drafts a fix, a human confirms, the agent re-checks

SparkDBA • Thank you

Team GPTOPS